

架构候选方案对比表

对比维度	方案A：分层架构 (Layered)	方案B：MVC 架构	方案C：微服务架构 (Microservices)	方案D：事件驱动架构 (Event-Driven)
开发复杂度	较低。采用接口层、业务层、数据层标准分层结构，模块调用关系清晰，全部为本地进程内调用，无分布式与异步复杂问题。	低。遵循控制器、模型、视图分离模式，开发模式直观统一，适合快速构建界面与业务流程，学习与开发成本低。	高。需按业务域拆分为多个独立服务，涉及服务注册、网关路由、远程调用、权限传递等额外工作，整体链路复杂。	较高。基于事件发布与订阅实现模块解耦，需设计事件模型、消息通道与消费逻辑，异步流程会提升开发与调试难度。
可维护性	良好。层级边界清晰，功能模块相对独立，便于定位问题与代码修改；遵循规范可长期保持结构整洁，维护成本可控。	一般。结构简单易懂，但业务规模扩大后控制器易膨胀、逻辑易混杂，长期迭代会导致耦合度上升。	优秀。各服务职责单一、相互隔离，单个模块变更不影响全局；但分布式环境下问题追踪与运维成本更高。	优秀。模块间通过事件松耦合，变更局部流程不影响其他模块；但事件链路复杂后追踪与排查难度会明显提升。
可扩展性	有限。仅支持整体系统扩容，无法针对高负载模块做独立扩展，弹性能力较弱。	较弱。系统整体耦合度高，扩展单一功能通常需要修改多处关联代码，难以实现局部能力弹性扩容。	很强。可按业务压力对单个服务独立扩容，灵活应对流量波动，横向扩展能力极强。	强。生产者与消费者解耦，可独立扩展事件生产或消费能力，适合高并发、流量波动明显的场景。
性能	良好。全部为本地方法调用，无网络、序列化开销，接口响应稳定，可满足常规用户访问与操作需求。	优秀。请求链路短，无远程调用损耗，单机环境下响应速度最快，适合轻量化、高交互场景。	存在一定损耗。服务间通过网络通信，会产生额外延迟；高并发下吞吐量更高，但单次请求延迟高于单体。	中等。异步化可提升吞吐量，但事件投递、序列化与持久化会带来一定延迟，实时性略低于同步调用。
团队学习成本	低。只需掌握标准分层开发思想与常规框架用法，无分布式、异步等高阶知识要求，上手快。	极低。模式经典易懂，资料丰富，团队成员可快速理解并参与开发，适配小型团队快速推进。	较高。需学习服务治理、远程调用、分布式事务、链路追踪等技术，技术栈复杂，学习成本高。	较高。需理解事件模型、消息可靠性、幂等性、异常重试机制，对团队设计与调试能力要求更高。

对比维度	方案A：分层架构 (Layered)	方案B：MVC 架构	方案C：微服务架构 (Microservices)	方案D：事件驱动架构 (Event-Driven)
10周内可行性	极高。架构稳定、环境简单、开发效率高，可在规定周期内完成全部核心功能开发、调试与集成，风险可控。	较高。开发速度最快，可快速实现界面与基础流程，但业务规模扩大后扩展性与维护性受限。	极低。服务拆分、环境搭建、联调与分布式问题排查会占用大量时间，4人团队难以按期交付稳定系统。	较低。事件设计、消息可靠性、异步流程调试会显著延长开发周期，短期内容易出现流程不一致问题。